Lab 39 — Multifactor Authentication with TOTP

> **SY0-701 Objective 4.6** — Given a scenario, implement and maintain identity and access management.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~15 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: enrol a user in TOTP, prove that a password alone no longer authenticates, and see replay protection and backup codes in action.

You'll build: a TOTP-enrolled account, a two-factor PAM service (password plus time-based code) tested with `pamtester`, and an SSH configuration that demands both factors for that user.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 38 is helpful (PAM basics) but not required; tools installed in step 1.

Background

- **MFA** combines factors from **different** categories: something you know (password, PIN), something you have (token, phone, security key), something you are (biometrics), and the attributes somewhere you are and something you do. A password plus a security question is two steps from one category, not MFA.
- A **soft token** is an authenticator app on a phone; a **hard token** is a dedicated device such as a key fob. Both generate one-time passwords from a shared secret. **TOTP** (RFC 6238) derives the code from the time in 30-second steps; **HOTP** (RFC 4226) uses an event counter.
- **Security keys** (FIDO2/WebAuthn) hold a private key bound to the website origin, which makes them phishing-resistant; SMS codes are the weakest second factor because of SIM swapping and interception.
- Good OTP implementations add **replay protection** (a used code is refused), **rate limiting** and one-time **backup (scratch) codes** for account recovery.

Steps

1. Install the tools

The Google Authenticator PAM module is open source and works with any TOTP app. `oathtool` computes codes the same way a phone app does, so you can test without a phone.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y libpam-google-authenticator oathtool qrencode pamtester openssh-server
```

2. Create a user and enrol a TOTP secret

Run the enrolment as the user so the secret file is owned by them and readable only by them. The flags select time-based codes (`-t`), disallow code reuse (`-d`), rate-limit to 10 attempts per 30 seconds, accept one step of clock skew either side (`-w 3`) and create five backup codes.

```
id mfauser >/dev/null 2>&1 || useradd -m -s /bin/bash mfauser
echo 'mfauser:Labs-Pass-2026' | chpasswd
sudo -u mfauser google-authenticator -t -d -f -r 10 -R 30 -w 3 -e 5 -q -s
/home/mfauser/.google_authenticator
ls -l /home/mfauser/.google_authenticator
```

```
-r-------- 1 mfauser mfauser 132 Oct 28 10:15 /home/mfauser/.google_authenticator
```

3. Inspect the shared secret and the provisioning QR code

The first line is the Base32 shared secret; the quoted lines are options; the 8-digit lines are backup codes. The QR code simply encodes an `otpauth://` URI, which is why a photo of an enrolment QR code is as sensitive as the secret itself.

```
SECRET=$(head -1 /home/mfauser/.google_authenticator)
echo "Shared secret (Base32): $SECRET"
grep '^"' /home/mfauser/.google_authenticator
URI="otpauth://totp/C628-Lab:mfauser?secret=${SECRET}&issuer=C628-Lab&digits=6&period=30"
echo "$URI"
qrencode -t ANSIUTF8 "$URI"
```

Optionally scan the QR code with Google Authenticator, Microsoft Authenticator or Aegis; the app will show the same codes as the next step.

4. Generate codes like an authenticator app (TOTP versus HOTP)

Both sides hold the same secret and compute HMAC-SHA1 over a moving factor. For TOTP the factor is the current 30-second time step; for HOTP it is a counter.

```
SECRET=$(head -1 /home/mfauser/.google_authenticator)
oathtool --totp -b "$SECRET"
oathtool --totp -b -w 2 "$SECRET"
oathtool --hotp -b -c 0 -w 2 "$SECRET"
date -u +%s | awk '{print "Seconds left in this time step:", 30 - ($1 % 30)}'
```

5. Build a two-factor PAM service

The `forward_pass` option asks one prompt: type the password immediately followed by the six-digit code. The module checks the code, then passes the password to pam_unix, so **both** factors must be correct.

```
cat > /etc/pam.d/mfa-lab <<'EOF'
# Factor 1 (know) + factor 2 (have): password immediately followed by the 6-digit TOTP
code
auth    required    pam_google_authenticator.so forward_pass
auth    required    pam_unix.so use_first_pass
account required    pam_unix.so
EOF
cat /etc/pam.d/mfa-lab
```

6. Prove that one factor is not enough

A stolen password alone fails, and so does the right password with a guessed code.

```
echo 'Labs-Pass-2026' | pamtester mfa-lab mfauser authenticate || echo "REJECTED: password
only"
echo 'Labs-Pass-2026000000' | pamtester mfa-lab mfauser authenticate || echo "REJECTED:
password + wrong code"
```

```
pamtester: Authentication failure
REJECTED: password only
pamtester: Authentication failure
REJECTED: password + wrong code
```

7. Log in with both factors, then test replay protection and a backup code

The correct password plus the current code succeeds. Replaying the same code a second later is refused because of `-d`, which defeats an attacker who shoulder-surfs or relays a code. A backup code works exactly once.

```
SECRET=$(head -1 /home/mfauser/.google_authenticator)
CODE=$(oathtool --totp -b "$SECRET")
echo "Labs-Pass-2026${CODE}" | pamtester mfa-lab mfauser authenticate && echo "ACCEPTED:
password + current code"
echo "Labs-Pass-2026${CODE}" | pamtester mfa-lab mfauser authenticate || echo "REJECTED:
replay of a used code"
SCRATCH=$(grep -E '^[0-9]{8}$' /home/mfauser/.google_authenticator | head -1)
echo "Labs-Pass-2026${SCRATCH}" | pamtester mfa-lab mfauser authenticate && echo
"ACCEPTED: backup code (now burned)"
grep -cE '^[0-9]{8}$' /home/mfauser/.google_authenticator
```

```
pamtester: successfully authenticated
ACCEPTED: password + current code
pamtester: Authentication failure
REJECTED: replay of a used code
pamtester: successfully authenticated
ACCEPTED: backup code (now burned)
4
```

8. Enforce MFA for SSH logins

Apply the same module to SSH for this user only. `nullok` lets users who have not enrolled keep logging in during roll-out, which is a deliberate (and temporary) exception to document.

```
cp -n /etc/pam.d/sshd /root/sshd.pam.bak
grep -q pam_google_authenticator /etc/pam.d/sshd || echo 'auth required
pam_google_authenticator.so nullok' >> /etc/pam.d/sshd
echo 'KbdInteractiveAuthentication yes' > /etc/ssh/sshd_config.d/60-mfa.conf
grep -q 'Match User mfauser' /etc/ssh/sshd_config || printf '\nMatch User mfauser\n
AuthenticationMethods keyboard-interactive\n' >> /etc/ssh/sshd_config
sshd -t && systemctl restart ssh && echo "sshd config valid and reloaded"
```

To try it interactively, open a second terminal tab and log in; you will be asked for the password and then the verification code (generate it with the `oathtool` command from step 4):

```
ssh -o PreferredAuthentications=keyboard-interactive mfauser@127.0.0.1
(mfauser@127.0.0.1) Password:
(mfauser@127.0.0.1) Verification code:
```

Test it

```
SECRET=$(head -1 /home/mfauser/.google_authenticator)
echo 'Labs-Pass-2026' | pamtester mfa-lab mfauser authenticate || echo "PASS: password
alone rejected"
sleep 31
echo "Labs-Pass-2026$(oathtool --totp -b "$SECRET")" | pamtester mfa-lab mfauser
authenticate && echo "PASS: both factors accepted"
sshd -T -C user=mfauser,host=localhost,addr=127.0.0.1 | grep -i authenticationmethods
```

Expected: the password alone is rejected, a fresh code plus the password is accepted, and `sshd -T` shows `authenticationmethods keyboard-interactive` for mfauser.

Troubleshooting

Symptom	Fix
Correct password and code still fail	The clock is off, or you reused a code. Check `timedatectl`, wait for the next 30-second step, and remember `-d` refuses any code already used.
`Failed to read "/home/mfauser/.google_authenticator"` in the journal	The secret file must be owned by the user and not group/world readable. Re-run step 2 with `sudo -u mfauser`.
`sshd -t` reports an error about `Match`	The Match block must be the last section of `sshd_config`. Restore with `cp /root/sshd.pam.bak /etc/pam.d/sshd` and edit the file so nothing follows the Match block.

Challenge

Create a second user, enrol them with HOTP instead of TOTP (`google-authenticator -c ...`), and use `oathtool --hotp -c N` to log in three times. Explain what happens to the counter and why HOTP codes stay valid until used.

Reflection

An attacker phishes both the password and the current TOTP code through a fake login page and relays them within 30 seconds. Which MFA implementation from objective 4.6 would have stopped this, and why?

Exam tips

- TOTP in an app is "something you have" (the enrolled device), not "something you know", even though you type it.
- Phishing-resistant MFA = FIDO2/WebAuthn security keys or passkeys; SMS OTP is the weakest second factor.

- Biometric accuracy terms: FAR (false acceptance), FRR (false rejection), CER (crossover, where FAR = FRR, lower is better).
- Push fatigue (MFA bombing) is mitigated by number matching and rate limits.

References

- RFC 6238 TOTP — https://datatracker.ietf.org/doc/html/rfc6238
- RFC 4226 HOTP — https://datatracker.ietf.org/doc/html/rfc4226
- google-authenticator-libpam — https://github.com/google/google-authenticator-libpam
- NIST SP 800-63B authenticators — https://pages.nist.gov/800-63-3/sp800-63b.html
- WebAuthn demo — https://webauthn.io
